

1|

操作计划追踪系统 — 设计文档 v2

2|

1. 背景与目标

3|

工作台(Workbench)依赖用户手动填写“明日计划”，数据源经过人工筛选，不适合追踪系统本身的判断质量。

4|

v2 目标： 复盘页面每天17:00实时计算的 `trading_plan`（每日交易计划）中的个股操作数据，自动追踪次日涨跌表现。

5|

核心问题

6|

7|

- 系统说“中继买点” → 第二天实际涨了还是跌了？各买点类型成功率多少？

8|

- 系统说“上涨趋势·上行” → 这个结构+阶段判断准不准？

9|

- 主线板块的股票 vs 非主线，成功率差别多大？

10|

- 持仓操作建议的准确率 vs 关注买入建议的准确率？

11|

12|

2. 数据源

13|

数据来源

14|

不再依赖工作台文件（`workbench/{date}.json`），改为直接从复盘实时计算结果中提取：

15|

```
from backend.services.review_service import compute_review_real_time
16|data = compute_review_real_time(date_str)
17|trading_plan = data['trading_plan']
```

18|

`trading_plan` 包含两个个股操作列表：

19|

holdings_action[] — 持仓股操作建议 | 字段 | 类型 | 示例 | |——|——|——| | `stock` | `str` | “国际复材(301526)” | | `action` | `str` | “执行突破买点” / “持有不动” / “卖出” | | `reason` | `str` | “上涨趋势·上行” (structure·stage) | | `priority` | `str` | “高” / “中” / “低” | | `stop_loss` | `float` | 止损价（如果有） | | `stop_loss_pct` | `float` | 止损百分比 | | `change` | `float` | 当日涨跌幅 |

20|

buy_priority[] — 关注买入信号 | 字段 | 类型 | 示例 | |——|——|——| | `name` | `str` | “广钢气体” | | `code` | `str` | “688548” | | `sector` | `str` | “电子化学品” | | `buy_point` | `str` | “中继买点” / “涨停回踩” / “趋势回踩EMAS” | | `is_main` | `bool` | 是否主线板块 | | `profit_model1` | `bool` | 是否盈利模式1 | | `trend_stock` | `bool` | 是否趋势股 | | `structure` | `str` | “上涨趋势” | | `stage` | `str` | “上行” | | `change` | `float` | 当日涨跌幅 | | `stop_loss` | `float` | 止损价 | | `stop_loss_pct` | `float` | 止损百分比 | | `priority` | `int` | 优先级排序 |

21|

注意：数据源是实时计算的 `compute_review_real_time(date_str)`，不依赖任何存档文件。

22|

3. 数据模型与存储

23|

存储方案：SQLite

24|

使用 Python 标准库 `sqlite3`，零外部依赖，适合多维度聚合统计。

25|

数据库路径：`{DATA_DIR}/private/plan_tracking.db`

26|

表结构

27|

```
CREATE TABLE IF NOT EXISTS plan_records (  
  28|   id              INTEGER PRIMARY KEY AUTOINCREMENT,  
  29|   date            TEXT    NOT NULL,      -- 计划日期 '2026-05-28'  
  30|   code            TEXT    NOT NULL,      -- 股票代码  
  31|   name            TEXT,                  -- 股票名称  
  32|  
  33|   -- 来源分类  
  34|   source          TEXT,      -- 'holdings_action' 或 'buy_priority'  
  35|   action          TEXT,      -- 系统建议操作  
  36|   reason          TEXT,      -- 结构·阶段：'上涨趋势·上行'  
  37|   structure       TEXT,      -- 结构：上涨趋势/区间震荡/下降趋势  
  38|   stage           TEXT,      -- 阶段：上行/缩量整理/区底  
  39|   buy_point       TEXT,      -- 买点类型  
  40|   is_main         INTEGER, -- 0/1 是否主线  
  41|   priority        TEXT,      -- 高/中/低  
  42|  
  43|   -- 止损  
  44|   stop_loss       REAL,      -- 止损价  
  45|   stop_loss_pct   REAL,      -- 止损百分比  
  46|  
  47|   -- 计划当天行情  
  48|   plan_close      REAL,      -- 当日收盘价  
  49|  
  50|   -- 次日追踪  
  51|   next_date       TEXT,      -- 下一个交易日  
  52|   next_open       REAL,      -- 次日开盘  
  53|   next_close      REAL,      -- 次日收盘  
  54|   next_high       REAL,      -- 次日最高  
  55|   next_low        REAL,      -- 次日最低  
  56|  
  57|   -- 计算结果  
  58|   change_pct      REAL,      -- 次日涨跌幅  
  59|   max_gain        REAL,      -- 盘中最大涨幅  
  60|   max_loss        REAL,      -- 盘中最大跌幅  
  61|   hit_stop_loss   INTEGER, -- 0/1 盘中是否跌破止损  
  62|   result          TEXT,      -- success / failure / flat / pending / no_data  
  63|  
  64|   -- 用户标记  
  65|   executed        INTEGER, -- NULL=未知 0=未执行 1=已执行  
  66|   user_note       TEXT,      -- 用户备注  
  67|  
  68|   -- 时间戳  
  69|   created_at      TEXT,      -- 首次写入时间  
  70|   updated_at      TEXT,      -- 最后更新时间
```

```
71|
72|     UNIQUE(date, code)
73|);
74|
75|-- 统计维度的索引
76|CREATE INDEX idx_records_date      ON plan_records(date);
77|CREATE INDEX idx_records_result    ON plan_records(result);
78|CREATE INDEX idx_records_source    ON plan_records(source);
79|CREATE INDEX idx_records_buy_point ON plan_records(buy_point);
80|CREATE INDEX idx_records_structure ON plan_records(structure, stage);
81|CREATE INDEX idx_records_is_main   ON plan_records(is_main);
```

82|

多维度统计

83|

```
-- 成功率统计（任意维度组合）
84|SELECT buy_point, count(*) as total,
85|       sum(CASE WHEN result='success' THEN 1 ELSE 0 END) as wins,
86|       round(avg(change_pct), 2) as avg_chg
87|FROM plan_records
88|WHERE result IN ('success','failure')
89|GROUP BY buy_point;
90|
91|-- 按结构·阶段
92|SELECT structure, stage, count(*), avg(change_pct)
93|FROM plan_records GROUP BY structure, stage;
94|
95|-- 按是否主线
96|SELECT is_main, count(*) as total,
97|       sum(CASE WHEN result='success' THEN 1 ELSE 0 END) as wins,
98|       round(avg(CASE WHEN result='success' THEN 1.0 ELSE 0.0 END) * 100, 1) as rate
99|FROM plan_records WHERE result IN ('success','failure')
100|GROUP BY is_main;
101|
102|-- 交叉聚合：主线+买点
103|SELECT is_main, buy_point, count(*) as total,
104|       sum(CASE WHEN result='success' THEN 1 ELSE 0 END) as wins
105|FROM plan_records WHERE result IN ('success','failure')
106|GROUP BY is_main, buy_point;
```

107|

成功/失败判定规则

108|

同 v1，保持不变：

109|

买入方向（含 `buy_priority + holdings_action` 中买入建议）： | 条件 | 判定 | |—|—|—| | 次日收盘 > 当日收盘 +0.5% | ✓
success | | 次日收盘 < 当日收盘 -0.5% | ✗ failure | | 介于 ±0.5% | — flat | | 无次日数据 | ☒ pending |

110|

卖出方向： | 条件 | 判定 | |—|—|—| | 次日收盘 < 当日收盘 -0.5% | ✓ success（卖对了） | | 次日收盘 > 当日收盘 +0.5% | ✗ failure（卖飞了） |

111|

持有/观察方向： 不参与成功率统计，仅展示次日涨跌。

112|

4. 系统架构

113|

```
114| | compute_review_real_time(date_str) |
115| |     → trading_plan.holdings_action (持仓操作) |
116| |     → trading_plan.buy_priority (关注买入) |
117| | | | |
118| | | | |
119| | | | ▼
120| | | | |
121| | | | plan_tracking_service.py |
122| | | | - scan_plan_data(): 从 review 提取个股操作数据 |
123| | | | - compute_tracking(): 计算次日涨跌并存入 SQLite |
124| | | | - get_tracking(): 读取 + 日期筛选 + 多维度统计 |
125| | | | 存储: plan_tracking.db (SQLite) |
126| | | | |
127| | | | |
128| | | | ▼
129| | | | |
130| | | | API 路由 (plan_tracking.py) |
131| | | | GET /api/plan-tracking 获取追踪结果 |
132| | | | POST /api/plan-tracking/annotate 标记执行状态 |
133| | | | POST /api/plan-tracking/refresh 强制重新计算 |
134| | | | |
```

135|

5. API 设计

136|

GET /api/plan-tracking

137|

```
{
  138|   "plans": [
  139|     {
  140|       "id": 1,
  141|       "date": "2026-05-28",
  142|       "code": "301526",
  143|       "name": "国际复材",
  144|       "source": "holdings_action",
  145|       "action": "执行突破买点",
  146|       "reason": "上涨趋势·上行",
  147|       "structure": "上涨趋势",
  148|       "stage": "上行",
  149|       "buy_point": "突破买点",
  150|       "is_main": 1,
  151|       "priority": "高",
  152|       "plan_close": 12.50,
  153|       "next_date": "2026-05-29",
  154|       "next_close": 13.20,
  155|       "change_pct": 5.6,
  156|       "result": "success",
  157|       "executed": null
  158|     }
  159|   ],
  160|   "summary": {
  161|     "total_plans": 22,
  162|     "success": 14,
  163|     "failure": 6,
  164|     "flat": 2,
  165|     "pending": 0,
  166|     "success_rate": 70.0,
  167|     "avg_gain_pct": 3.2,
  168|     "avg_loss_pct": -2.1,
  169|     "best_gain": 8.5,
  170|     "worst_loss": -4.3,
```

```
171|   "win_loss_ratio": 1.52
172| },
173| "by_buy_point": {
174|   "中继买点": { "total": 8, "success": 6, "failure": 1, "flat": 1, "rate": 85.7 },
175|   "涨停回踩": { "total": 5, "success": 3, "failure": 2, "flat": 0, "rate": 60.0 },
176|   "趋势回踩EMA5": { "total": 4, "success": 2, "failure": 1, "flat": 1, "rate": 66.7 }
177| },
178| "by_structure": {
179|   "上涨趋势": { "total": 12, "success": 9, "failure": 2, "flat": 1, "rate": 81.8 },
180|   "区间震荡": { "total": 8, "success": 4, "failure": 3, "flat": 1, "rate": 57.1 }
181| },
182| "by_is_main": {
183|   "1": { "total": 10, "success": 8, "failure": 1, "flat": 1, "rate": 88.9 },
184|   "0": { "total": 12, "success": 6, "failure": 5, "flat": 1, "rate": 54.5 }
185| },
186| "by_source": {
187|   "holdings_action": { "total": 12, "success": 7, "failure": 4, "flat": 1 },
188|   "buy_priority": { "total": 10, "success": 7, "failure": 2, "flat": 1 }
189| },
190| "suggestions": [
191|   {
192|     "type": "warning",
193|     "dimension": "buy_point",
194|     "category": "缩量整理",
195|     "rate_current": 33.3,
196|     "count": 3,
197|     "message": "条件「缩量整理」3次仅33%成功率"
198|   }
199| ],
200| "last_updated": "2026-05-30T10:00:00"
201|}
```

202|

POST /api/plan-tracking/annotate

203|

标记计划执行状态。

204|

POST /api/plan-tracking/refresh

205|

强制重新计算未完成的追踪。

206|

6. 前端页面

207|

复用 v1 的 PlanTracking.tsx，适配新数据模型： - 统计卡片行（成功率/平均盈利/盈亏比）保持不变 - 按买点类型分组（替代原来的条件类型） - 按结构·阶段分组（新增） - 按是否主线分组（新增） - 自动建议逻辑更新为新维度

208|

7. 归类维度总结

209| 210| 211| 212| 213| 214| 215| 216| 217| 218| 219| 220| 221| 222| 223| 224| 225| 226| 227| 228| 229| 230| 231| 232| 233| 234| 235| 236| 237| 238| 239| 240| 241| 242| 243| 244| 245| 246| 247| 248| 249|

维度	来源字段	统计示例
买点类型	buy_point	中继买点 85%
结构	structure	上涨趋势 81%
结构+阶段	structure, stage	上涨趋势·上行 85%

是否主线	is_main	主线 89%
数据来源	source	持仓操作 vs 关注买入
操作方向	action	买入建议 70%

250|

8. 不做 (v1 范围内)

251|

252|

- 第3天/第5天追踪 — v2 再扩展

253|

- 自动标记”已执行” — 需交易系统接入

254|

- 实盘成交价追踪 — 无券商API

255|

- 板块/大盘同期对比 — v2

256|

257|

9. 文件清单

258|

新增/修改:		
259	docs/plan-tracking-design.md	– 本设计文档 (v2更新)
260	server/backend/services/plan_tracking_service.py	– 重写: 数据源从 workbench-review, 存储从 JSON-SQLite
261	server/backend/api/plan_tracking.py	– 适配新数据模型
262	server/backend/tests/test_plan_tracking.py	– 重写测试 (TDD)
263		
264	数据:	
265	data/private/plan_tracking.db	– SQLite数据库 (自动生成, 取代 plan_tracking.json)

266|

10. 分支策略

267|

268|

- 分支: feature/plan-tracking

269|

- TDD 开发

270|

- 完成后 PR 合并 master

271|

272|